GEEK ROOM · GLOBAL AI HACKATHON · BIFROST TRACK

StickyRoute

One conversation, one AI provider.

An open-source fix for Bifrost, the AI gateway: weighted routing that keeps every conversation on the same provider, while keeping your traffic split exact.

TEAM MATRIXX

Akash · Rahul · Kanishk · Ronak

FIXES MAXIMHQ/BIFROST#5409 · RD #0108

Hi Everyone,

As we approach the Global AI Hackathon submission deadline, we want to clarify the project submission options once again.

You have three options:

Single Track: Build one project using a problem statement from either the Bifrost or Tavily track.

Combined Project: Build one project incorporating one problem statement from each track.

Two Separate Projects: Build two different projects, one for each track.

Prizes:

Bifrost Track: $1,000 in cash prizes for winners.

Tavily Track: 60,000 Tavily credits for winners.

Double Win: A team can win prizes from both tracks!

Certificates: Every team that successfully submits a project will receive participation certificates.

START HERE

The whole project in 30 seconds

1

What is Bifrost?

An open-source AI gateway. Apps send every AI request to Bifrost, and Bifrost forwards it to a provider such as Claude, GPT or Gemini.

2

What was broken?

Teams split traffic, say 70% Claude / 30% GPT. Bifrost re-picks the provider on every message, so one chat keeps jumping between models.

3

What did we build?

One opt-in switch per routing rule, sticky_sessions. Each conversation picks a provider once and stays there. The 70/30 split still holds.

Result on a real Bifrost gateway: **974 of 1,000 conversations switched provider before. 0 of 1,000 after.**

FOR READERS NEW TO BIFROST

Background: what weighted routing does



Why teams split traffic

Cost, rate limits, a backup provider, or trying a new model on a slice of users.

What a "conversation" is

Many messages sharing one history. Coding agents and chat apps send a session ID with each one.

The catch

The split was applied to each message, not each conversation. That is issue #5409.

THE PROBLEM

Bifrost splits per message, not per conversation

One real conversation through Bifrost today, rule set to 70% A / 30% B

msg 1	msg 2	msg 3	msg 4	msg 5	msg 6	msg 7	msg 8	msg 9	msg 10
B	A	A	A	B	A	B	A	B	A

7 provider switches in 10 messages

Threads break

Providers differ in message format and tool calling, so a mid-thread switch can fail outright.

Cache is thrown away

Each switch lands on a cold prompt cache, so the long shared history is billed again at full price.

Voice changes mid-chat

Users see the assistant's style and answers shift halfway through one chat.

WHY EXISTING FEATURES DID NOT HELP

Weighted rules defeat Bifrost's own affinity

1

Message 1

Rule draws A. Session affinity remembers: this session uses A.

2

Message 2

Rule re-rolls and draws B. The list it offers no longer contains A.

3

Binding dropped

Affinity cannot keep a provider it was not offered, so the session moves.

Bifrost already has session affinity. It never gets a chance, because the weighted draw happens first. So the fix belongs in the routing rule, not in a new store.

OUR SOLUTION

The fix: one opt-in switch per routing rule

```
"routing_rules": [{
  "name": "claude-gpt-split",
  "sticky_sessions": true,
  "targets": [
    {"provider": "anthropic", "weight": 0.7},
    {"provider": "openai",    "weight": 0.3}
  ]
}]
```

Also a switch in the Bifrost web UI, in 11 languages.

1. **Same conversation, same provider**

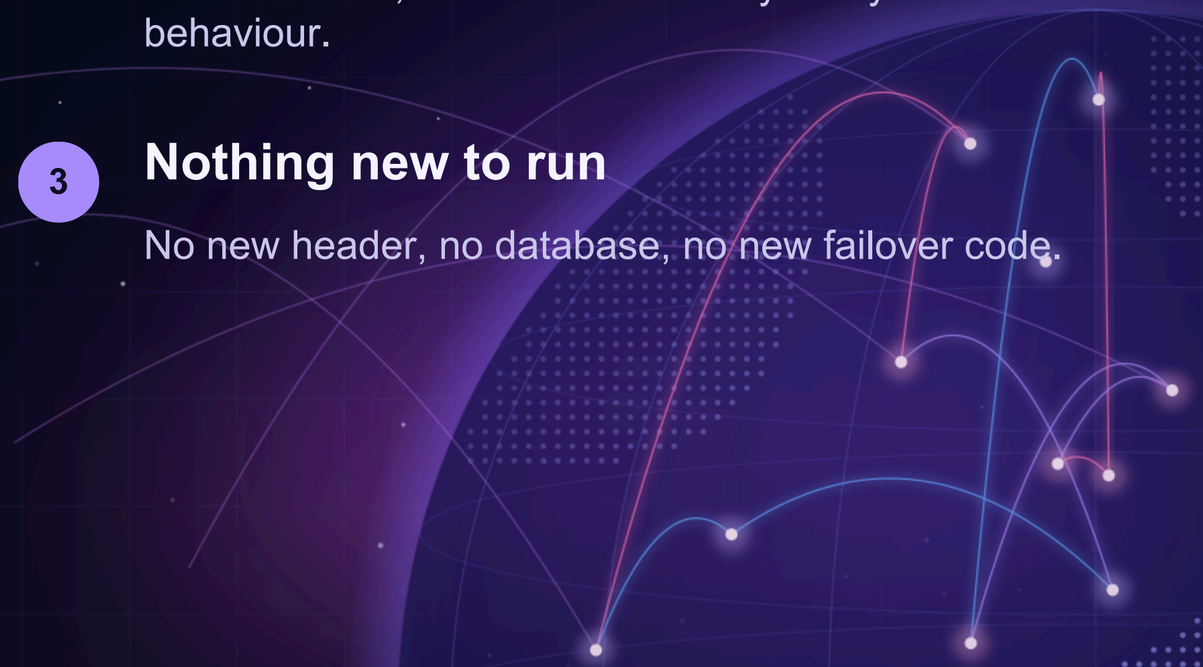
   Uses the session ID Bifrost already reads (x-bf-session-id; Claude Code, Codex and OpenCode send one).

2. **Nothing else changes**

   No session ID, or switch off: exactly today's behaviour.

3. **Nothing new to run**

   No new header, no database, no new failover code.



HOW IT WORKS

How it picks: weighted rendezvous hashing

Think of it as every conversation holding a fixed lottery ticket. The ticket is computed from its session ID, so it never changes, and bigger weights win more often.

```
score(provider) = weight / −ln( hash(session, rule, provider) )
```

the provider with the highest score answers the whole conversation

70.08%

share for a 70% target over 100,000 sessions: the split stays exact

Stable

same session ID → same provider, on every gateway node, with no shared state

Minimal moves

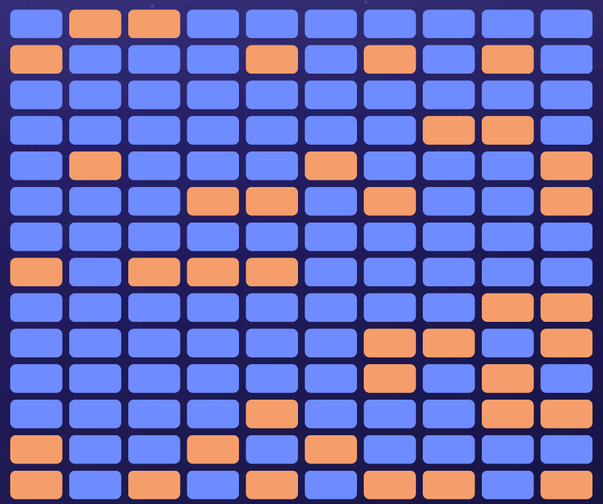
remove a provider and only its own sessions move; target order does not matter

Why not hash(session) mod total? One weight change would reshuffle almost every live session.

PROOF

Measured on a real Bifrost gateway: 974 → 0

Bifrost today



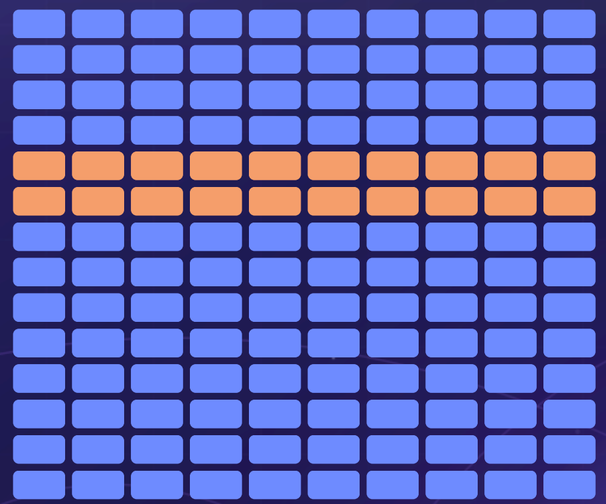
974

of 1,000 conversations switched provider

3.76 switches per conversation

Provider A Provider B

With sticky_sessions



0

of 1,000 conversations switched provider

Split held: 72.7% / 27.3%

Provider A Provider B

Each row is a conversation, each cell a message, coloured by who answered (first 14 shown). Gateway built from our branch, 70/30 rule, 1,000 conversations × 10 messages.
Source: bench/results/routing-proof.json

LIVE RUN

And with a real AI provider

9 / 10

conversations switched model
Bifrost today

0 / 10

conversations switched model
with sticky_sessions

120 / 120

real requests answered
no errors

Setup: Gemini free tier through the same gateway, 70/30 across two Gemini models, 10 conversations × 6 messages.
Honest limit: this proves routing on real traffic. It does not measure cache savings; that needs a paid two-provider run.

Source: bench/results/free-gemini-70-30.json

ENGINEERING

Built like an upstream contribution

+651 −23

lines across 28 files

12

new routing tests, plus config-hash and API round-trip tests

11

UI languages for the new switch

5 / 5

CodeRabbit review comments answered

Engine, migration, API, config / OpenAPI / Helm schemas and docs in one change

Existing rules keep their config hash: no backfill migration, no spurious resyncs

Respects x-bf-session-affinity: off, and scopes the sticky key per caller

Test suites pass, also under the race detector; UI builds with 0 type errors

IMPACT

Why it matters

Reliability

Multi-turn agents and tool-calling chats stop breaking when traffic is split.

Cost

Each conversation keeps a warm prompt cache on one provider instead of paying for its history again.

Safe rollouts

Try a new model on 10% of conversations, not 10% of messages, and compare like with like.

Cache savings are the expected effect of staying on one provider; we have not yet measured them with paid providers.

STATUS

Ready for upstream review

Upstream PR	github.com/maximhq/bifrost/pull/9908
Live demo	akashjhajharia123-ui.github.io/stickyroute
Code and results	github.com/akashjhajharia123-ui/stickyroute



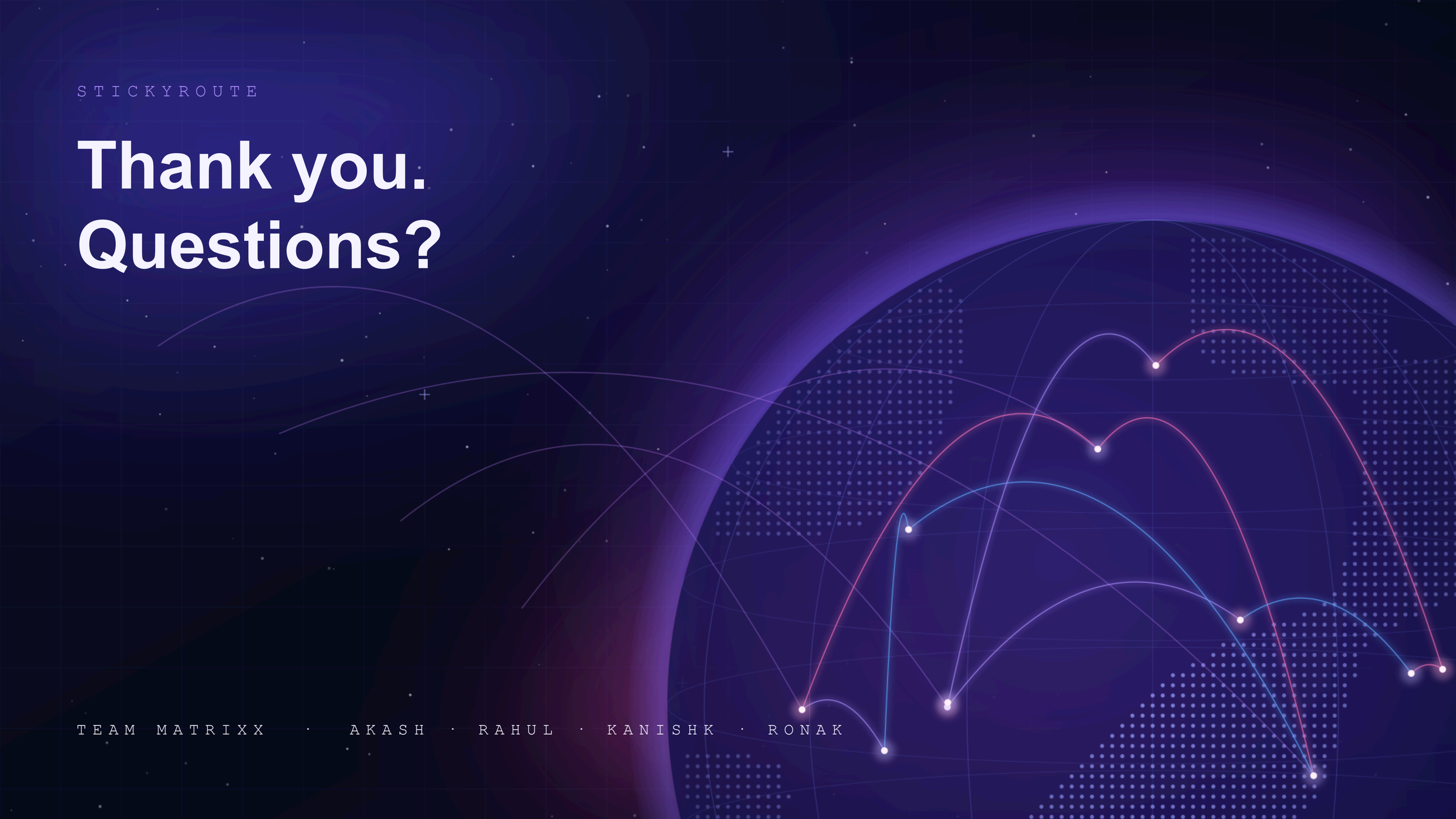
NEXT

Paid Claude + GPT run to measure cache savings, an optional selector hook, a per-team stickiness TTL.

TEAM MATRIXX · AKASH · RAHUL · KANISHK · RONAK

STICKYROUTE

Thank you.
Questions?



TEAM MATRIXX · AKASH · RAHUL · KANISHK · RONAK